Practical Work 1
In422 : Real-Time Information Systems

Student :
Eliott MOSKOWICZ

Niveau :
Aéro 4 Promotion 2027

Tuesday 28th April 2026
School year 2025-2026
M. SINGH

Introduction

This project focuses on the implementation and analysis of a real-time task system, specifically evaluating the execution behaviour and schedulability of a given task set.

The core components of the assignment include :

- WCET Estimation : Experimentally obtaining the execution time distribution (Min, Max, Q1, Q2, Q3) for task τ_1 to determine its WCET (C_1).
- Schedulability Analysis : Verifying if the provided task set, consisting of seven tasks with specific periods and computation times, is schedulable within its hyperperiod.
- Algorithm Optimization : Developing a non-preemptive scheduler in Python, C, or C++ aimed at eliminating deadline misses while minimizing total waiting time across all jobs.
- Alternative Scheduling : Exploring a secondary scheduling strategy that prioritizes the minimization of waiting time, even if it necessitates a deadline miss for task τ_5.

1 Tasks Schedulability

The given Task Set is :

Task	C	T_i
τ_1	C_1	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

FIGURE 1 – Task Set

To determine if the task set is schedulable, we must first calculate the total Processor Utilization (U). A real-time system is only schedulable if the CPU has enough capacity to handle the workload within the given time frames. The system is considered potentially schedulable if $U \leq 1$.

The utilization of a task τ_i is defined as the ratio of its execution time (C_i) to its period (T_i) :

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

With our given task set we need :

$$U = \frac{C1}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} \leq 1$$

$$\iff C_1 \leq 3.625\, ms$$

To ensure a complete analysis, we study the system over its Hyperperiod, which is the Least Common Multiple (LCM) of all task periods (T_i).

$$\text{Hyperperiod (H)} : \text{LCM}(10, 20, 40, 80) = 80\, ms$$

This 80 ms window represents the minimum time after which the schedule repeats itself exactly.

2 WCET Analysis of Task τ_1

This section outlines the experimental procedure used to determine the timing characteristics of the first task, τ_1, which is responsible for calculating the product of large, randomly generated numbers.

2.1 Experimental Setup

To obtain realistic execution data, the computational logic was implemented in C to ensure high performance and low-level resource management.

The core logic is defined in `multiplication.c`, utilizing a loop of 10,000 iterations to create a stable and measurable computational load, ensuring the execution time was long enough to be captured accurately by the system clock.
To prevent the CPU from "cheating" by remembering previous results (cached), large random numbers were generated as inputs for every single test.

To ensure the results were statistically significant and to capture the true Worst-Case, this process was repeated 50,000 times. This massive sample size helps filter out temporary slowdowns caused by the operating system.

2.2 Statistical Results and WCET Determination

After executing the benchmark, we collected 50,000 timing samples to understand the task's behavior. The results from the experiment show a clear distribution of timings, summarized in the screenshot below :

```
Metric                      | Value
---------------------------------------------
WCET (Worst Case)           |   2.4618 ms
WCET + 20% margin           |   2.9542 ms
Average Timing              |   0.0302 ms
Minimum Timing              |   0.0115 ms
---------------------------------------------
First Quantile (Q1)         |   0.0239 ms
Second Quantile (Median)    |   0.0248 ms
Third Quantile (Q3)         |   0.0301 ms
=============================================
```

FIGURE 2 – Scheduling results with all tasks on time

We find $C_1 = 2.9542\,ms$ which respect our schedulability condition ($C_1 < 3.625\,ms$). For the rest of this project, we will use $C_1 = 2.9542\,ms$.

With this value we can re-compute the CPU Utilization (U) :

$$U = \frac{2.9542}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80} \leq 1$$

$$U = 0.93292$$

It represents the CPU Utilization (U = 93.29%).

3 Case 1 : Standard NP-EDF (No deadline miss)

This section presents the primary scheduling results where the main priority is ensuring that every job completes before its deadline.

3.1 Scheduling Strategy

The scheduler uses a non-preemptive Earliest Deadline First (EDF) algorithm. The logic follows three strict rules :

- Causality : A job can only start if the current time is greater than or equal to its release time (r).
- Priority : Among all jobs that have arrived and are ready to execute, the job with the earliest absolute deadline (d) is chosen.
- Non-Preemption : Once a job starts, it occupies the processor for its full execution time (C) without interruption, even if a more urgent job arrives during that window.

3.2 Response time verification

To guarantee the system is schedulable, we must verify the response time R_{ij}^n for every job. The response time is the total time from the moment a job is released until it finishes its execution.
According to the assignment, the response time for the nth job of task i is calculated using the following recursive formula :

$$R_{ij}^n = C_{ij}^n + \max(R_{ij}^{n-1} - (a_{ij}^n - a_{ij}^{n-1}), 0)$$

Where :

- C_{ij}^n : The execution time of the current job.
- R_{ij}^{n-1} : The response time of the previous job.
- $a_{ij}^n - a_{ij}^{n-1}$: The time elapsed between the arrival of the current job and the previous one (the period T_i).

This mathematical proof confirms that our non-preemptive EDF schedule is valid and that no deadlines are missed.

```
--- VERSION 1: NO DEADLINE MISSES ---
Job      | Arrival  | Start    | End      | Time Rep | Max   | Schedulable ?
-----------------------------------------------------------------------
T1_J1    | 0.0      | 0.00000  | 2.95420  | 2.95     | 10.0  | YES
T2_J1    | 0.0      | 2.95420  | 5.95420  | 5.95     | 10.0  | YES
T3_J1    | 0.0      | 5.95420  | 7.95420  | 7.95     | 20.0  | YES
T4_J1    | 0.0      | 7.95420  | 9.95420  | 9.95     | 20.0  | YES
T5_J1    | 0.0      | 9.95420  | 11.95420 | 11.95    | 40.0  | YES
T1_J2    | 10.0     | 11.95420 | 14.90840 | 4.91     | 10.0  | YES
T2_J2    | 10.0     | 14.90840 | 17.90840 | 7.91     | 10.0  | YES
T6_J1    | 0.0      | 17.90840 | 19.90840 | 19.91    | 40.0  | YES
T7_J1    | 0.0      | 19.90840 | 22.90840 | 22.91    | 80.0  | YES
T1_J3    | 20.0     | 22.90840 | 25.86260 | 5.86     | 10.0  | YES
T2_J3    | 20.0     | 25.86260 | 28.86260 | 8.86     | 10.0  | YES
T3_J2    | 20.0     | 28.86260 | 30.86260 | 10.86    | 20.0  | YES
T4_J2    | 20.0     | 30.86260 | 32.86260 | 12.86    | 20.0  | YES
T1_J4    | 30.0     | 32.86260 | 35.81680 | 5.82     | 10.0  | YES
T2_J4    | 30.0     | 35.81680 | 38.81680 | 8.82     | 10.0  | YES
T1_J5    | 40.0     | 40.00000 | 42.95420 | 2.95     | 10.0  | YES
T2_J5    | 40.0     | 42.95420 | 45.95420 | 5.95     | 10.0  | YES
T3_J3    | 40.0     | 45.95420 | 47.95420 | 7.95     | 20.0  | YES
T4_J3    | 40.0     | 47.95420 | 49.95420 | 9.95     | 20.0  | YES
T5_J2    | 40.0     | 49.95420 | 51.95420 | 11.95    | 40.0  | YES
T1_J6    | 50.0     | 51.95420 | 54.90840 | 4.91     | 10.0  | YES
T2_J6    | 50.0     | 54.90840 | 57.90840 | 7.91     | 10.0  | YES
T6_J2    | 40.0     | 57.90840 | 59.90840 | 19.91    | 40.0  | YES
T1_J7    | 60.0     | 60.00000 | 62.95420 | 2.95     | 10.0  | YES
T2_J7    | 60.0     | 62.95420 | 65.95420 | 5.95     | 10.0  | YES
T3_J4    | 60.0     | 65.95420 | 67.95420 | 7.95     | 20.0  | YES
T4_J4    | 60.0     | 67.95420 | 69.95420 | 9.95     | 20.0  | YES
T1_J8    | 70.0     | 70.00000 | 72.95420 | 2.95     | 10.0  | YES
T2_J8    | 70.0     | 72.95420 | 75.95420 | 5.95     | 10.0  | YES
```

FIGURE 3 – Response Time Analysis

As we can see, no deadline are missed and this task set is fully schedulable.

3.3 Results from the non-preemptive EDF algorithm

The goal of this firts algorithm is to ensure that every task is executed with no deadline miss.

```
--- VERSION 1: NO DEADLINE MISSES ---
Job      | Arrival  | Start    | End      | Deadline | Status
-----------------------------------------------------------------------
T1_J1    | 0.0      | 0.00000  | 2.95420  | 10.0     | OK
T2_J1    | 0.0      | 2.95420  | 5.95420  | 10.0     | OK
T3_J1    | 0.0      | 5.95420  | 7.95420  | 20.0     | OK
T4_J1    | 0.0      | 7.95420  | 9.95420  | 20.0     | OK
T5_J1    | 0.0      | 9.95420  | 11.95420 | 40.0     | OK
T1_J2    | 10.0     | 11.95420 | 14.90840 | 20.0     | OK
T2_J2    | 10.0     | 14.90840 | 17.90840 | 20.0     | OK
T6_J1    | 0.0      | 17.90840 | 19.90840 | 40.0     | OK
T7_J1    | 0.0      | 19.90840 | 22.90840 | 80.0     | OK
T1_J3    | 20.0     | 22.90840 | 25.86260 | 30.0     | OK
T2_J3    | 20.0     | 25.86260 | 28.86260 | 30.0     | OK
T3_J2    | 20.0     | 28.86260 | 30.86260 | 40.0     | OK
T4_J2    | 20.0     | 30.86260 | 32.86260 | 40.0     | OK
T1_J4    | 30.0     | 32.86260 | 35.81680 | 40.0     | OK
T2_J4    | 30.0     | 35.81680 | 38.81680 | 40.0     | OK
T1_J5    | 40.0     | 40.00000 | 42.95420 | 50.0     | OK
T2_J5    | 40.0     | 42.95420 | 45.95420 | 50.0     | OK
T3_J3    | 40.0     | 45.95420 | 47.95420 | 60.0     | OK
T4_J3    | 40.0     | 47.95420 | 49.95420 | 60.0     | OK
T5_J2    | 40.0     | 49.95420 | 51.95420 | 80.0     | OK
T1_J6    | 50.0     | 51.95420 | 54.90840 | 60.0     | OK
T2_J6    | 50.0     | 54.90840 | 57.90840 | 60.0     | OK
T6_J2    | 40.0     | 57.90840 | 59.90840 | 80.0     | OK
T1_J7    | 60.0     | 60.00000 | 62.95420 | 70.0     | OK
T2_J7    | 60.0     | 62.95420 | 65.95420 | 70.0     | OK
T3_J4    | 60.0     | 65.95420 | 67.95420 | 80.0     | OK
T4_J4    | 60.0     | 67.95420 | 69.95420 | 80.0     | OK
T1_J8    | 70.0     | 70.00000 | 72.95420 | 80.0     | OK
T2_J8    | 70.0     | 72.95420 | 75.95420 | 80.0     | OK
-----------------------------------------------------------------------
Total Waiting Time: 180.08 ms
Total Idle Time: 1.32 ms
```

FIGURE 4 – Scheduling results with no deadline miss

As we can see in the Figure 3 above, the task set is schedulable. All 29 jobs meet their deadlines. The metrics for this first algorithm are :

- **Total Waiting Time :** 180.08 ms
- **Total Idle Time :** 1.32 ms

4 Case 2 : τ_5 is allowed to miss its deadline

4.1 Scheduling Strategy

As required, a second algorithm was created in which the task τ_5 is allowed to exceed its deadline to prioritize the overall efficiency of the system. In this scenario, we modified the scheduling priority to favor tasks with shorter execution times and higher frequencies whenever τ_5 is ready to execute. While the standard EDF algorithm focuses on the most urgent deadline, this modified version treats τ_5 as a "low-priority" background task.

```
--- SCENARIO 2 : MINIMAL WAITING (T5 ALLOWED TO MISS) ---
Job      | Arrival  | Start    | End      | Deadline | Status
-----------------------------------------------------------------
T1_J1    | 0.0      | 0.00000  | 2.95420  | 10.0     | OK
T2_J1    | 0.0      | 2.95420  | 5.95420  | 10.0     | OK
T3_J1    | 0.0      | 5.95420  | 7.95420  | 20.0     | OK
T4_J1    | 0.0      | 7.95420  | 9.95420  | 20.0     | OK
T6_J1    | 0.0      | 9.95420  | 11.95420 | 40.0     | OK
T1_J2    | 10.0     | 11.95420 | 14.90840 | 20.0     | OK
T2_J2    | 10.0     | 14.90840 | 17.90840 | 20.0     | OK
T7_J1    | 0.0      | 17.90840 | 20.90840 | 80.0     | OK
T1_J3    | 20.0     | 20.90840 | 23.86260 | 30.0     | OK
T2_J3    | 20.0     | 23.86260 | 26.86260 | 30.0     | OK
T3_J2    | 20.0     | 26.86260 | 28.86260 | 40.0     | OK
T4_J2    | 20.0     | 28.86260 | 30.86260 | 40.0     | OK
T1_J4    | 30.0     | 30.86260 | 33.81680 | 40.0     | OK
T2_J4    | 30.0     | 33.81680 | 36.81680 | 40.0     | OK
T5_J1    | 0.0      | 36.81680 | 38.81680 | 40.0     | OK
T1_J5    | 40.0     | 40.00000 | 42.95420 | 50.0     | OK
T2_J5    | 40.0     | 42.95420 | 45.95420 | 50.0     | OK
T3_J3    | 40.0     | 45.95420 | 47.95420 | 60.0     | OK
T4_J3    | 40.0     | 47.95420 | 49.95420 | 60.0     | OK
T6_J2    | 40.0     | 49.95420 | 51.95420 | 80.0     | OK
T1_J6    | 50.0     | 51.95420 | 54.90840 | 60.0     | OK
T2_J6    | 50.0     | 54.90840 | 57.90840 | 60.0     | OK
T5_J2    | 40.0     | 57.90840 | 59.90840 | 80.0     | OK
T1_J7    | 60.0     | 60.00000 | 62.95420 | 70.0     | OK
T2_J7    | 60.0     | 62.95420 | 65.95420 | 70.0     | OK
T3_J4    | 60.0     | 65.95420 | 67.95420 | 80.0     | OK
T4_J4    | 60.0     | 67.95420 | 69.95420 | 80.0     | OK
T1_J8    | 70.0     | 70.00000 | 72.95420 | 80.0     | OK
T2_J8    | 70.0     | 72.95420 | 75.95420 | 80.0     | OK

Total Waiting Time (V2): 184.9848 ms
Total Idle Time (V2): 1.3206 ms
```

FIGURE 5 – Scheduling results with τ_5 being low-priority

As observed in Figure 4 above, the results are :

- **Total Waiting Time :** 184.9848 ms
- **Total Idle Time :** 1.3206 ms

We can note that despite our intuition, no deadline are missed in this second scenario. Also, the total waiting time is actually 4 ms longer in this second case which was supposed to minimize the waiting time by categorizing the task τ_5 as low-priority. The total idle time is also slightly longer.

5 Computational complexity

The computational complexity of the developed scheduling algorithm is analyzed as a function of the number of tasks n and the total number of jobs N generated within the 80 ms hyperperiod. The initial job generation process, which iterates through each task to calculate its instances based on its period T_i, possesses a complexity of $O(N)$, where $N = \sum(Hyperperiod/T_i)$. For the scheduling loop, the algorithm evaluates every job release by filtering the remaining list for ready jobs and sorting them by their absolute deadlines to satisfy the Earliest Deadline First (EDF) strategy. In the worst-case scenario, this repeated search and sorting across the entire hyperperiod results in an overall complexity of $O(N^2)$. However, given the small number of tasks ($n = 7$) and the short duration of the hyperperiod, this complexity remains well within the performance limits of the execution environment, ensuring negligible overhead during the scheduling process.

6 Conclusion

The goal of this project was to design two scheduler for a high-utilization task set. By implementing a C function called inside a Python file, we successfully determined a mesurable Worst Case Execution Time for the task τ_1 (C1 2.9542 ms), which placed our total processor utilization at approximately 93.29%.

We managed to prove that the given task set was completely schedulable and that every task met its deadline regardless of the algorithm.

The comparison between the two scheduling scenarios revealed a counter-intuitive outcome. Deliberately treating τ_5 as a low-priority task—even with the flexibility to potentially miss its deadline—did not improve the system's overall performance. Instead, the total waiting time increased by 4 ms, proving that the standard EDF priority logic

remains the most efficient choice for this specific configuration. Additionally, the total idle time was slightly longer in this second case, which indicates that sub-optimal task ordering in a non-preemptive environment can lead to inefficient gaps in processor usage rather than clearing the queue faster. Remarkably, despite these timing shifts, no deadlines were actually missed in the second scenario, and every job continued to satisfy the response time verification condition where $R_{ij} \leq T_i$.